Universidade Federal do Rio Grande – FURG
Sistemas Distribuídos
Rio Grande/RS, Brasil
Graduação em Engenharia de Computação

CLIENTE/SERVIDOR COM WEB SERVICE: TRAVELAI

Membros do grupo: Andrey Vinicius Santos Souza – 164402; Guilherme Rodrigues Estrella – 164391.

1 MOTIVAÇÃO E DEFINIÇÃO DO PROBLEMA

Ao planejar uma viagem, o usuário costuma consultar sites diferentes para descobrir a localização de uma cidade, a previsão do tempo, dados do país e a cotação da moeda local em relação ao real brasileiro (BRL). Essa consulta fragmentada dificulta a comparação das informações. O TravelAI reúne os dados em uma interface web e trata as respostas de serviços externos antes de apresentá-las.

2 ARQUITETURA DO SISTEMA E REGRAS DE FUNCIONAMENTO

O navegador executa a interface React e consulta diretamente quatro serviços públicos. A camada `src/services` valida as respostas JSON com Zod e as converte em modelos internos de `src/types`, apresentados pelos componentes em vez de expor os JSONs brutos. O roteiro e as perguntas contextuais usam uma função serverless em `/api/gemini`, que consulta o Google Gemini sem expor a chave de API ao navegador.

2.1 Tecnologias utilizadas

React 19, TypeScript, Vite e Tailwind CSS compõem a interface. A Fetch API realiza as requisições; um cliente HTTP comum aplica timeout e converte falhas em erros conhecidos. Vitest e Testing Library cobrem a lógica e os componentes. Playwright testa a navegação no navegador com respostas simuladas. A função serverless usa a API REST do Gemini.

2.2 Web services integrados

Tabela 1: Serviços e informações usadas no TravelAI

Serviço	Informações apresentadas ao usuário
Open-Meteo Geocoding	Cidade, país, código ISO, coordenadas e fuso horário.
Open-Meteo Forecast	Condições atuais e previsão de cinco dias.
countries.dev	Nome, bandeira, capital, população, idiomas e moedas do país.
Frankfurter	Cotação indicativa de BRL para a moeda local e data de referência.
Google Gemini	Roteiro de viagem e resposta contextual a uma pergunta.

2.3 Fluxo das requisições

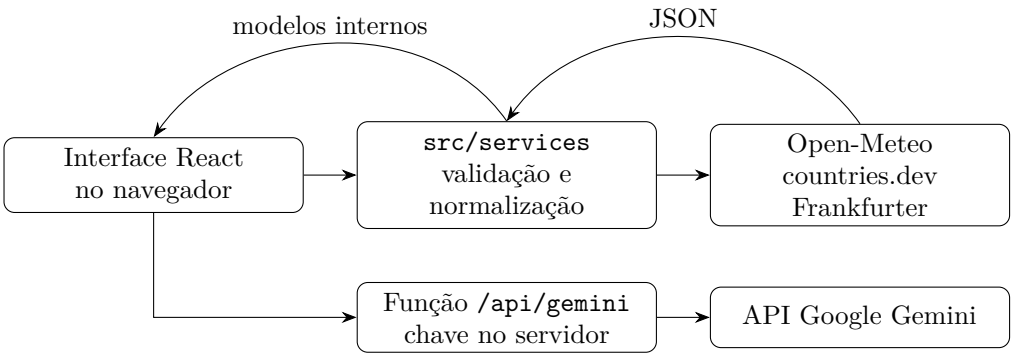


Figura 1: Serviços públicos acessados pelo navegador; Gemini acessado pela função serverless.

Depois da busca de uma cidade, o usuário seleciona um resultado; entre nomes repetidos, a aplicação prioriza o país esperado e centros urbanos. A seleção fornece coordenadas e fuso para a previsão, e o código ISO do país para consultar os dados nacionais. A cotação parte de BRL e é solicitada apenas quando a moeda do destino é compatível. Clima, país e câmbio têm estados independentes de carregamento, sucesso e falha; uma nova busca cancela as requisições anteriores. O navegador recebe os modelos internos após validação com Zod e apresenta campos formatados, não o JSON bruto.

A Vercel foi escolhida por hospedar os arquivos estáticos do Vite e a função serverless no mesmo domínio HTTPS. Isso oferece acesso remoto pelo navegador e mantém a chave Gemini no ambiente do servidor, sem exigir a administração de um servidor próprio. A interface envia as solicitações de IA como JSON para `/api/gemini`; a função restringe método e origem, limita o corpo, valida a entrada e solicita JSON estruturado. A resposta é validada novamente e, no roteiro, a quantidade e a sequência dos dias também são conferidas antes de retornar ao frontend. Em falhas transitórias ou timeout, a função tenta um modelo alternativo.

3 ACESSO À APLICAÇÃO E EXECUÇÃO LOCAL

Aplicação online: `https://travelai-furg.vercel.app`. A página foi publicada na Vercel e respondeu HTTP 200; a avaliação pode ser feita pelo navegador, sem instalação. **Código-fonte:** `https://github.com/dreyvinixz/TripScope`. Em 25 de outubro de 2026, os fluxos Gemini de pergunta contextual e geração de roteiro responderam HTTP 200. Quando o modelo principal retornou HTTP 503, o fallback `gemini-3.5-flash-lite` concluiu a solicitação.

Para desenvolvimento local, são necessários Node.js 20 ou superior e npm. Na raiz do repositório, executar `npm ci` e `npm run dev`; o Vite informa o endereço local. As quatro APIs públicas não exigem chave. Para usar o Gemini, definir `GEMINI_API_KEY` no ambiente do servidor, com base em `.env.example`, e reiniciar o Vite. A chave não deve usar o prefixo `VITE_`.

Os comandos de verificação são `npm run lint`, `npm run typecheck`, `npm test`, `npm run build` e `npm run test:e2e`. Os testes unitários e E2E principais usam *fixtures* e *mocks*. `npm run test:contract` consulta os serviços públicos e depende da rede; a verificação final deve incluir a aplicação hospedada sem *mocks*.

4 DIFICULDADES ENFRENTADAS E SOLUÇÕES ADOTADAS

4.1 Indisponibilidade e formatos externos

Quatro serviços diferentes podem responder lentamente, falhar ou alterar seus formatos. O cliente HTTP converte erros em categorias conhecidas, e Zod verifica os JSONs antes da conversão para os modelos internos. O painel informa a falha do recurso afetado e mantém os demais cartões disponíveis.

4.2 Proteção da chave do Gemini

Uma chave inserida no código do cliente ficaria visível no navegador. A integração com Gemini foi isolada em `/api/gemini`. A função lê a chave do ambiente do servidor, valida a entrada e devolve apenas a resposta tratada para a interface.

4.3 Testes reproduzíveis

Testes automatizados que dependem de provedores externos podem falhar por rede ou limite de requisições. A suíte principal usa respostas simuladas e determinísticas; testes de contrato e a demonstração online verificam a integração real separadamente.

5 CONCLUSÃO

O **TravelAI** reúne informações de destinos vindas de serviços web distintos em uma interface útil ao viajante. A validação e a normalização das respostas reduzem o acoplamento com os formatos externos, enquanto os estados independentes permitem uso parcial quando um provedor falha. A integração com Gemini mantém a credencial fora do navegador, valida os dados e apresenta respostas estruturadas. Os testes online confirmaram a geração de roteiros e respostas contextuais, incluindo o uso de um modelo alternativo quando o principal ficou indisponível.